

运算

算术运算符

运算符	功能
+	(单目) 正
-	(单目) 负
*	(双目) 乘法
/	除法
%	取模
+	(双目) 加法
-	(双目) 减法

► 单目与双目运算符

算术运算符中有两个单目运算符（正、负）以及五个双目运算符（乘法、除法、取模、加法、减法），其中单目运算符的优先级最高。

其中取模运算符 `%` 意为计算两个整数相除得到的余数，即求余数。

而 `-` 为双目运算符时做减法运算符，如 $2-1$ ；为单目运算符时做负值运算符，如 -1 。

使用方法如下

```
op=x-y*z
```

得到的 `op` 的运算值遵循数学中加减乘除的优先规律，首先进行优先级高的运算，同优先级按运算的结合性运算，括号提高优先级。

算术运算中的类型转换

对于双目算术运算符，当参与运算的两个变量类型相同时，不发生 [类型转换](#)，运算结果将会用参与运算的变量的类型容纳，否则会发生类型转换，以使两个变量的类型一致。转换的规则参见 [类型转换](#)。

例如，对于一个整型（`int`）变量 和另一个双精度浮点型（`double`）类型变量：

- `x/3` 的结果将会是整型；
- `x/3.0` 的结果将会是双精度浮点型；
- `x/y` 的结果将会是双精度浮点型；
- `x*1/3` 的结果将会是整型；
- `x*1.0/3` 的结果将会是双精度浮点型；

位运算符

运算符	功能
~	逐位非
& (双目)	逐位与
	逐位或

运算符	功能
<code>^</code>	逐位异或
<code><<</code>	逐位左移
<code>>></code>	逐位右移

位操作的意义请参考 [位运算](#) 页面。需要注意的是，位运算符的优先级低于普通的算数运算符。

自增/自减 运算符

有时我们需要让变量进行增加 1（自增）或者减少 1（自减），这时自增运算符 `++` 和自减运算符 `--` 就派上用场了。

自增/自减运算符可放在变量前或变量后面，在变量前称为前缀，在变量后称为后缀，单独使用时前缀后缀无需特别区别，如果需要用到表达式的值则需注意，具体可看下面的例子。详细情况可参考 [引用](#) 介绍的例子部分。

```

1 i = 100;
2
3 op1 = i++; // op1 = 100, 先 op1 = i, 然后 i = i + 1
4
5 i = 100;
6
7 op2 = ++i; // op2 = 101, 先 i = i + 1, 然后赋值 op2
8
9 i = 100;
10
11 op3 = i--; // op3 = 100, 先赋值 op3, 然后 i = i - 1
12
13 i = 100;
14
15 op4 = --i; // op4 = 99, 先 i = i - 1, 然后赋值 op4

```

复合赋值运算符

复合赋值运算符实际上是表达式的缩写形式。可分为复合算术运算符 `+=`、`-=`、`*=`、`/=`、`%=` 和复合位运算符 `&=`、`|=`、`^=`、`<<=`、`>>=`。

例如，`op = op + 2` 可写为 `op += 2`，`op = op - 2` 可写为 `op -= 2`，`op = op * 2` 可写为 `op *= 2`。

条件运算符

条件运算符可以看作 `if` 语句的简写，`a ? b : c` 中如果表达式 `a` 成立，那么这个条件表达式的结果是 `b`，否则条件表达式的结果是 `c`。

比较运算符

运算符	功能
<code>></code>	大于
<code>>=</code>	大于等于
<code><</code>	小于
<code><=</code>	小于等于
<code>==</code>	等于
<code>!=</code>	不等于

其中特别需要注意的是要将等于运算符 `==` 和赋值运算符 `=` 区分开来，这在判断语句中尤为重要。

`if (op=1)` 与 `if (op==1)` 看起来类似，但实际功能却相差甚远。第一条语句是在对 `op` 进行赋值，若赋值为非 0 时为真值，表达式的条件始终是满足的，无法达到判断的作用；而第二条语句才是对 `op` 的值进行判断。

逻辑运算符

运算符 功能

<code>&&</code>	逻辑与
<code> </code>	逻辑或
<code>!</code>	逻辑非

```
1 Result = op1 && op2; // 当 op1 与 op2 都为真时则 Result 为真
2
3 Result = op1 || op2; // 当 op1 或 op2 其中一个为真时则 Result 为真
4
5 Result = !op1; // 当 op1 为假时则 Result 为真
```

内建的运算符 `&&` 和 `||` 进行短路求值（若在求值第一个操作数后结果已知，则不求值第二个），重载的运算符无此特性，并始终对两个操作数都进行求值。

逗号运算符

逗号运算符可将多个表达式分隔开来，被分隔开的表达式按从左至右的顺序依次计算，整个表达式的值是最后的表达式的值。逗号表达式的优先级在所有运算符中的优先级是 **最低** 的。

```
1 exp1, exp2, exp3; // 最后的值为 exp3 的运算结果。
2
3 Result = 1 + 2, 3 + 4, 5 + 6;
4 //得到 Result 的值为 3 而不是 11，因为赋值运算符 "="
5 //的优先级比逗号运算符高，先进行了赋值运算才进行逗号运算。
6
7 Result = (1 + 2, 3 + 4, 5 + 6);
8
9 // 若要让 Result 的值得到逗号运算的结果则应将整个表达式用括号提高优先级，此时
10 // Result 的值才为 11。
```

成员访问运算符

运算符	功能
<code>[]</code>	数组下标
<code>.</code>	对象成员
<code>&</code> （单目）	取地址/获取引用
<code>*</code> （单目）	间接寻址/解引用
<code>-></code>	指针成员

这些运算符用来访问对象的成员或者内存，除了最后一个运算符外上述运算符都可被重载。与 `&`，`*` 和 `->` 相关的内容请阅读 [指针](#) 和 [引用](#) 教程。这里还省略了两个很少用到的运算符 `.*` 和 `->*`，其具体用法可以参见 [C++ 语言手册](#)。

```
1 auto result1 = v[1]; // 获取v中下标为2的对象
2 auto result2 = p.q; // 获取p对象的q成员
3 auto result3 = p -> q; // 获取p指针指向的对象的q成员，等价于 (*p).q
4 auto result4 = &v; // 获取指向v的指针
5 auto result5 = *v; // 获取v指针指向的对象
```

C++ 运算符优先级总表

来自 [C++ 运算符优先级 - cppreference](#)，有修改。

运算符	描述	例子	可重载性
第一级别			
::	作用域解析符	Class::age = 2;	不可重载
第二级别			
++	后自增运算符	for (int i = 0; i < 10; i++) cout << i;	可重载
--	后自减运算符	for (int i = 10; i > 0; i--) cout << i;	可重载
type() type{}	强制类型转换	unsigned int a = unsigned(3.14);	可重载
()	函数调用	isdigit('1')	可重载
[]	数组数据获取	array[4] = 2;	可重载
.	对象型成员调用	obj.age = 34;	不可重载
->	指针型成员调用	ptr->age = 34;	可重载
第三级别（从右向左结合）			
++	前自增运算符	for (i = 0; i < 10; ++i) cout << i;	可重载
--	前自减运算符	for (i = 10; i > 0; --i) cout << i;	可重载
+	正号	int i = +1;	可重载
-	负号	int i = -1;	可重载
!	逻辑取反	if (!done) ...	可重载
~	按位取反	flags = ~flags;	可重载
(type)	C 风格强制类型转换	int i = (int) floatNum;	可重载
*	指针取值	int data = *intPtr;	可重载
&	值取指针	int *intPtr = &data	可重载
sizeof	返回类型内存	int size = sizeof floatNum; int size = sizeof(float);	不可重载
new	动态元素内存分配	long *pVar = new long; MyClass *ptr = new MyClass(args);	可重载
new []	动态数组内存分配	long *array = new long[n];	可重载
delete	动态析构元素内存	delete pVar;	可重载
delete []	动态析构数组内存	delete [] array;	可重载
第四级别			
.*	类对象成员引用	obj.*var = 24;	不可重载
->*	类指针成员引用	ptr->*var = 24;	可重载
第五级别			
*	乘法	int i = 2 * 4;	可重载
/	除法	float f = 10.0 / 3.0;	可重载
%	取余数（模运算）	int rem = 4 % 3;	可重载
第六级别			
+	加法	int i = 2 + 3;	可重载

运算符	描述	例子	可重载性
-	减法	<code>int i = 5 - 1;</code>	可重载
第七级别			
<<	位左移	<code>int flags = 33 << 1;</code>	可重载
>>	位右移	<code>int flags = 33 >> 1;</code>	可重载
第八级别			
<=>	三路比较运算符	<code>if ((i <=> 42) < 0) ...</code>	可重载
第九级别			
<	小于	<code>if (i < 42) ...</code>	可重载
<=	小于等于	<code>if (i <= 42) ...</code>	可重载
>	大于	<code>if (i > 42) ...</code>	可重载
>=	大于等于	<code>if (i >= 42) ...</code>	可重载
第十级别			
==	等于	<code>if (i == 42) ...</code>	可重载
!=	不等于	<code>if (i != 42) ...</code>	可重载
第十一级别			
&	位与运算	<code>flags = flags & 42;</code>	可重载
第十二级别			
^	位异或运算	<code>flags = flags ^ 42;</code>	可重载
第十三级别			
	位或运算	<code>flags = flags 42;</code>	可重载
第十四级别			
&&	逻辑与运算	<code>if (conditionA && conditionB) ...</code>	可重载
第十五级别			
	逻辑或运算	<code>if (conditionA conditionB) ...</code>	可重载
第十六级别（从右向左结合）			
? :	条件运算符	<code>int i = a > b ? a : b;</code>	不可重载
throw	异常抛出	<code>throw EClass("Message");</code>	不可重载
=	赋值	<code>int a = b;</code>	可重载
+=	加赋值运算	<code>a += 3;</code>	可重载
-=	减赋值运算	<code>b -= 4;</code>	可重载
*=	乘赋值运算	<code>a *= 5;</code>	可重载
/=	除赋值运算	<code>a /= 2;</code>	可重载
%=	模赋值运算	<code>a %= 3;</code>	可重载
<<=	位左移赋值运算	<code>flags <<= 2;</code>	可重载
>>=	位右移赋值运算	<code>flags >>= 2;</code>	可重载
&=	位与赋值运算	<code>flags &= new_flags;</code>	可重载
^=	位异或赋值运算	<code>flags ^= new_flags;</code>	可重载
=	位或赋值运算	<code>flags = new_flags;</code>	可重载
第十七级别			
,	逗号分隔符	<code>for (i = 0, j = 0; i < 10; i++, j++) ...</code>	可重载

需要注意的是，表中并未列出

`const_cast`、`static_cast`、`dynamic_cast`、`reinterpret_cast`、`typeid`、`sizeof...`、`noexcept` 及 `alignof` 等运算符，因为它们的使用形式与函数调用相同，不会出现歧义。

本页面最近更新：2024/2/28 12:39:47, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [aofall](#), [cmpute](#), [Enter-tainer](#), [Marcythm](#), [ouuan](#), [billchenchina](#), [countercurrent-time](#), [H-J-Granger](#), [lr1d](#), [leonfyr](#), [mgt](#), [NachtgeistW](#), [StudyingFather](#), [SukkaW](#), [Tiphereth-A](#), [ToUNVRSe](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用